ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 1ST YEAR

INTRODUCTION TO PROGRAMMING

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 1st year Introduction to Programming paper 1 · practice a.

Written by Academic Central for practice, following first-year Introduction to Programming syllabuses for engineering students at South African universities (Python). It is not a university memorandum.

How to mark: each ✓ is one mark. The code shown is one correct answer. Accept any working solution that gives the right result. Give marks for correct logic even with small syntax mistakes in QUESTIONS 5 and 6.

QUESTION 1

1.1 B ✓ (1)

1.2 B ✓ (1)

1.3 C ✓ (1)

1.4 B ✓ (1)

1.5 C ✓ (1)

1.6 B ✓ (1)

1.7 B ✓ (1)

1.8 A ✓ (1)

1.9 C ✓ (1)

1.10 C ✓ (1)

[10]

QUESTION 2

2.1 A compiler translates the whole program into machine code before it runs ✓; an interpreter translates and runs the program one statement at a time ✓ (2)

2.2 Syntax error: the code breaks the rules of the language, so it cannot run ✓, e.g. if x > 5 (no colon) ✓
Runtime error: the program starts but crashes while running ✓, e.g. 10 / 0 (ZeroDivisionError) ✓
Logic error: the program runs but gives the wrong result ✓, e.g. average = a + b / 2 instead of (a + b) / 2 ✓ (6)

2.3 A step-by-step set of clear instructions that solves a problem ✓
Any TWO: it must end after a finite number of steps; each step must be clear (unambiguous); it must give the correct output; it has defined input and output ✓✓ (3)

2.4 Any TWO: code can be reused instead of repeated; the program is easier to read and understand; each part can be tested and fixed on its own; work can be shared between programmers ✓✓ (2)

2.5 A local variable is created inside a function and can only be used inside that function ✓; a global variable is created outside all functions and can be used anywhere in the program ✓ (2)

[15]

QUESTION 3

3.1 11 3 ✓✓ (x = 5 + 2 × 3 = 11; y = 11 % 4 = 3) (2)

3.2 i = 1: −1; i = 2: 1; i = 3: 0; i = 4: 4; i = 5: 3 ✓✓✓ (4)
Output: 3 ✓

3.3 n: 20 → 10 → 5 → 2 → 1, count goes up 4 times ✓✓ (3)
Output: 1 4 ✓

3.4 [6, 16] ✓✓ 12 ✓ (3)

3.5 (5)

i	values[i]	big	pos
–	–	4	0
1	7	7	1
2	2	7	1
3	9	9	3
4	5	9	3

Starting values ✓ i = 1 and 2 ✓ i = 3 ✓ i = 4 ✓
Output: 9 3 ✓

3.6 OBO ✓ TOBOR ✓ 2 ✓ (3)

[20]

QUESTION 4

4.1 1. Missing colon after for t in temps: syntax error ✓, add a colon ✓ (8)
2. len(temp) uses a name that does not exist: runtime error (NameError) ✓, change it to len(temps) ✓
3. "Average: " + a float: runtime error (TypeError) ✓, use str(...) or an f-string ✓
4. An empty list makes len(temps) = 0: possible runtime error (ZeroDivisionError) ✓, check for an empty list first ✓

4.2 Corrected code ✓✓ (2)

```
def average(temps):
    if len(temps) == 0:
        return 0
    total = 0
    for t in temps:
        total = total + t
    return total / len(temps)

print("Average: " + str(average([20, 22, 24])))
```

[10]

QUESTION 5

5.1 def line ✓ formula ✓ return ✓ (3)

```
def c_to_f(c):
    return c * 9 / 5 + 32
```

5.2 Loop until −1 ✓ input converted to a number ✓ count ✓ smallest kept correctly ✓✓ too-close count ✓ −1 not counted ✓ three outputs ✓ (8)

```
count = 0
smallest = None
too_close = 0
reading = float(input("Reading (cm), -1 to stop: "))
while reading != -1:
    count += 1
    if smallest is None or reading < smallest:
        smallest = reading
    if reading < 10:
        too_close += 1
    reading = float(input("Reading (cm), -1 to stop: "))
print("Readings:", count)
print("Smallest:", smallest)
print("Too close:", too_close)
```

5.3 Numbers below 2 are not prime ✓ loop over possible factors ✓ test with % ✓ return False on a factor ✓ return True after the loop ✓ (5)

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True
```

5.4 One mark for each correct condition and return value ✓✓✓✓ (4)

```
def motor_state(speed):
    if speed < 0:
        return "ERROR"
    elif speed == 0:
        return "STOP"
    elif speed <= 30:
        return "SLOW"
    else:
        return "FAST"
```

5.5 New list ✓ correct loop range (len − 2) ✓ sum of 3 neighbours ✓ divide by 3 ✓ return the list ✓ (5)

```
def moving_average(data):
    result = []
    for i in range(len(data) - 2):
        result.append((data[i] + data[i + 1] + data[i + 2]) / 3)
    return result
```

5.6 class line ✓ __init__ with self.name and self.speed = 0 ✓ set_speed with the 0–100 check ✓ stop() ✓ object created and speed set ✓ (5)

```
class Motor:
    def __init__(self, name):
        self.name = name
        self.speed = 0

    def set_speed(self, s):
        if 0 <= s <= 100:
            self.speed = s
        else:
            print("Invalid speed")

    def stop(self):
        self.speed = 0

left = Motor("left wheel")
left.set_speed(60)
```

[30]

QUESTION 6

6.1 Open the file ✓ loop over lines ✓ strip and split on the comma ✓ append time ✓ append float(temp) ✓ (5)

```
times = []
temps = []
with open("log.txt") as f:
    for line in f:
        t, temp = line.strip().split(",")
        times.append(t)
        temps.append(float(temp))
```

6.2 Find the highest value ✓ find its position ✓ print the matching time ✓ (Answer for the data shown: 10:00) (3)

```
i = temps.index(max(temps))
print(times[i])
```

6.3 Open alarm.txt for writing ✓ loop over readings ✓ test > 24.0 ✓ write in the right format with a new line ✓ (4)

```
with open("alarm.txt", "w") as out:
    for i in range(len(temps)):
        if temps[i] > 24.0:
            out.write(times[i] + "," + str(temps[i]) + "\n")
```

6.4 The file is closed automatically, even if an error happens ✓ (1)

6.5 Python raises a FileNotFoundError and the program crashes ✓ (2)
Use try / except FileNotFoundError and print a friendly message ✓

```
try:
    with open("log.txt") as f:
        lines = f.readlines()
except FileNotFoundError:
    print("log.txt was not found")
```

[15]

TOTAL: 100

END OF MARKING GUIDELINE